1. Architecture

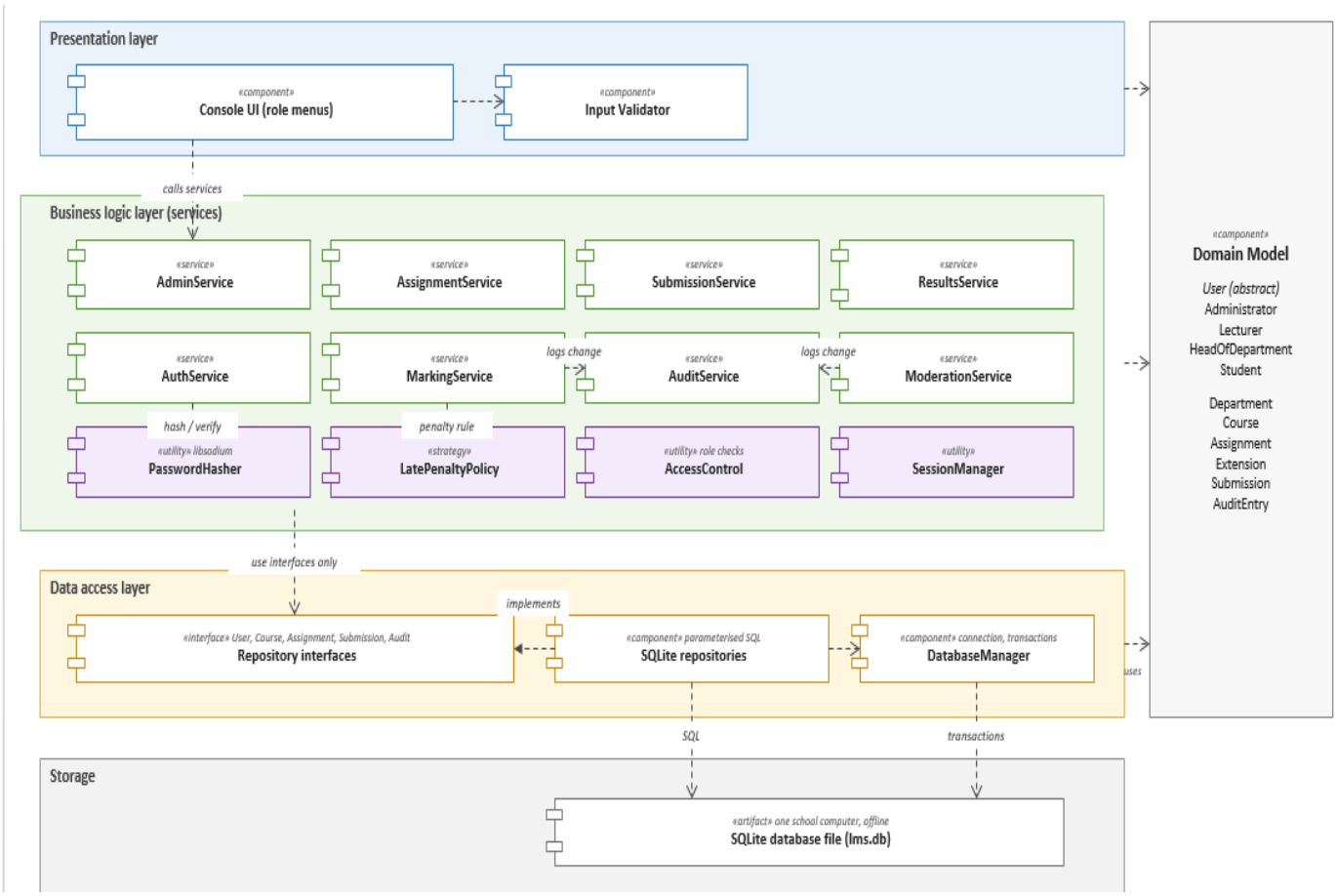
CampusNode: Moderated Learning Management System

a) Architectural style

The system uses a **layered architecture**. It has four layers and each layer only dependent on the layer below it:

- **Presentation:** the console menus. They show information and collect input.
- **Business logic:** services that apply the school rules (late penalty, extensions, moderation, audit log).
- **Data access:** repositories. This is the only code that writes SQL.
- **Storage:** the SQLite database file.

The **domain model** (User, Course, Assignment, Submission and so on) is shared by all layers. The business layer calls repository *interfaces*, not SQLite directly, so it does not depend on the database.



Each layer depends on the layer below it and business layer depends on repository interfaces, never on SQLite. Dashed arrows show dependencies. Every service calls Access Control before acting (role check, NFR5).

b) Design Rationale

Each part of the system has one job: show menus, apply school rules, or store data. A layered design keeps these apart. This makes the business rules easy to test without a database, keeps all SQL in one place, and is simple enough for one person to build in two weeks.

c) Alternatives Rejected

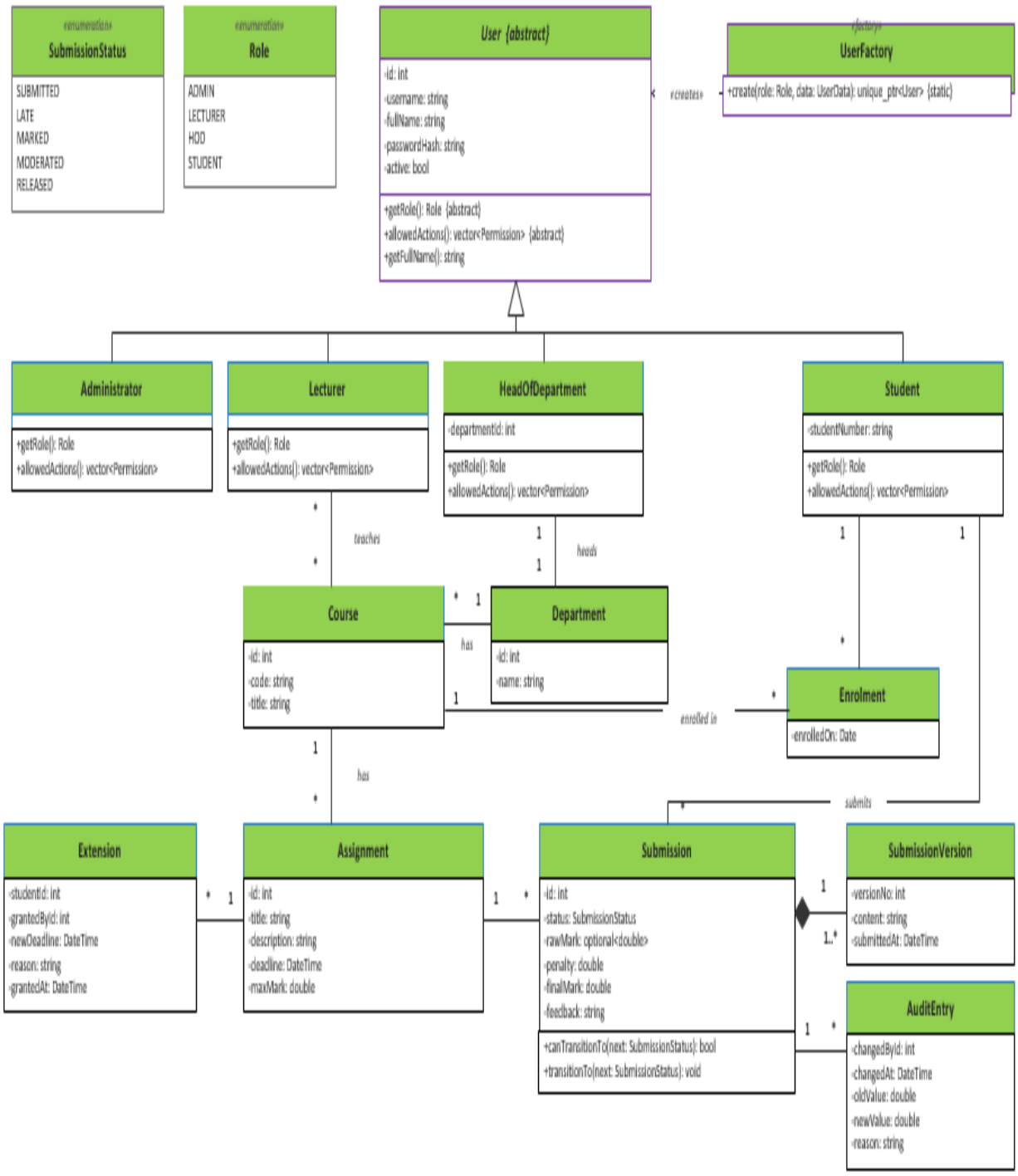
- **Model-View-Controller (MVC).** MVC splits a program into a model (the data and rules), a view (what the user sees) and a controller (which turns user actions into changes to the model). It works well for graphical and web programs, where several views of the same data must stay in step. For example, changing a mark might have to refresh a table, a chart and a summary at the same time. Our system is a console program that shows one screen at a time, so there are no linked views to keep in step, and MVC gives little benefit. It also has a practical risk. In small programs the controller tends to collect business rules such as the late penalty and the moderation rules, because it is the first place that receives the user's request. Those rules are the most important and most heavily tested part of the system. They need to sit in a separate layer that has no console or database code, so I can test them with automated tests and reach 70% coverage. I kept the useful idea from MVC, which is that the interface should be thin: the console menus only collect input and call services. If a graphical interface is added later, MVC would become a better choice, and the services could be reused unchanged.
- **Client-server with a database server (for example MySQL or PostgreSQL).** In this design the data lives in a server program, and many computers connect to it over the network. It is the usual choice when many people use a system at the same time. It was rejected for three reasons. First, we decided that the system runs on one school computer and works offline (A6 and BR8), so there is nobody to connect from another machine. Second, a database server must be installed, secured, updated and backed up on old school computers by a technician who is not a database expert, and the system would also need networking code and network security. That adds cost and risk (risks R6 and R9) for no benefit at this size. Third, SQLite already handles 800 students easily and needs no server. If the school later wants many computers to use the system, only the data access layer would need to change, because the services use repository interfaces and never SQLite directly.

2. CLASS DIAGRAM

a. Level Class Diagram

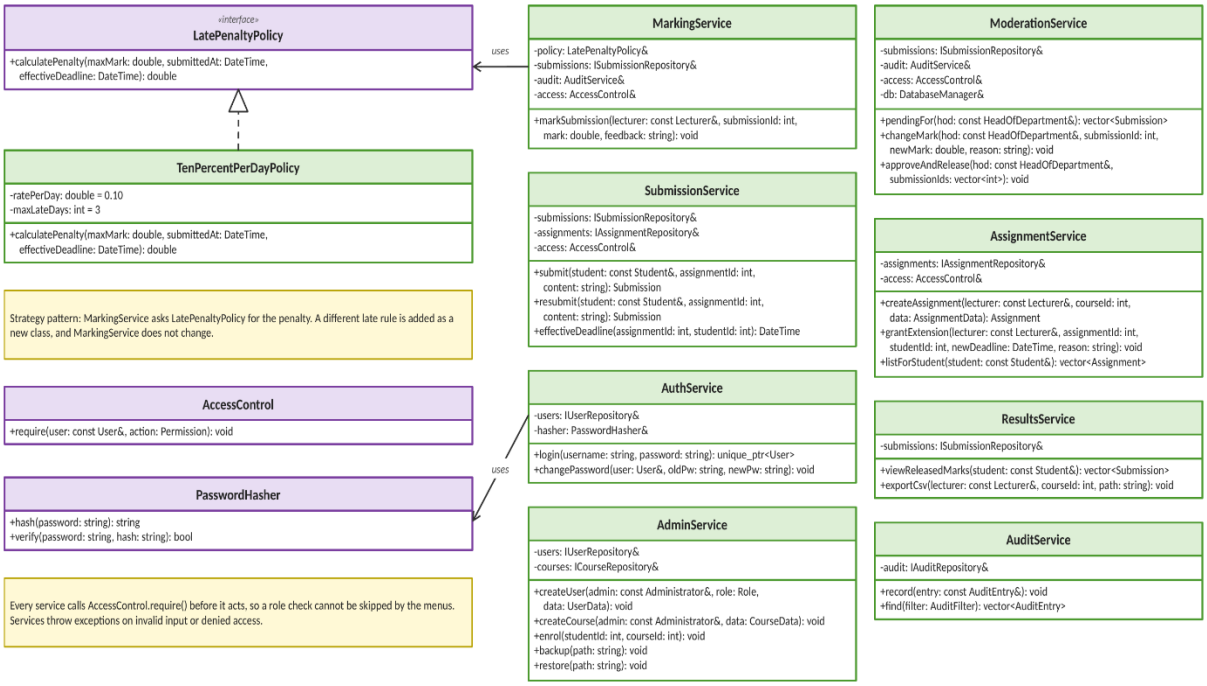
The class diagram is shown in three views so that each one is readable. Together they describe one design.

Domain model and user hierarchy

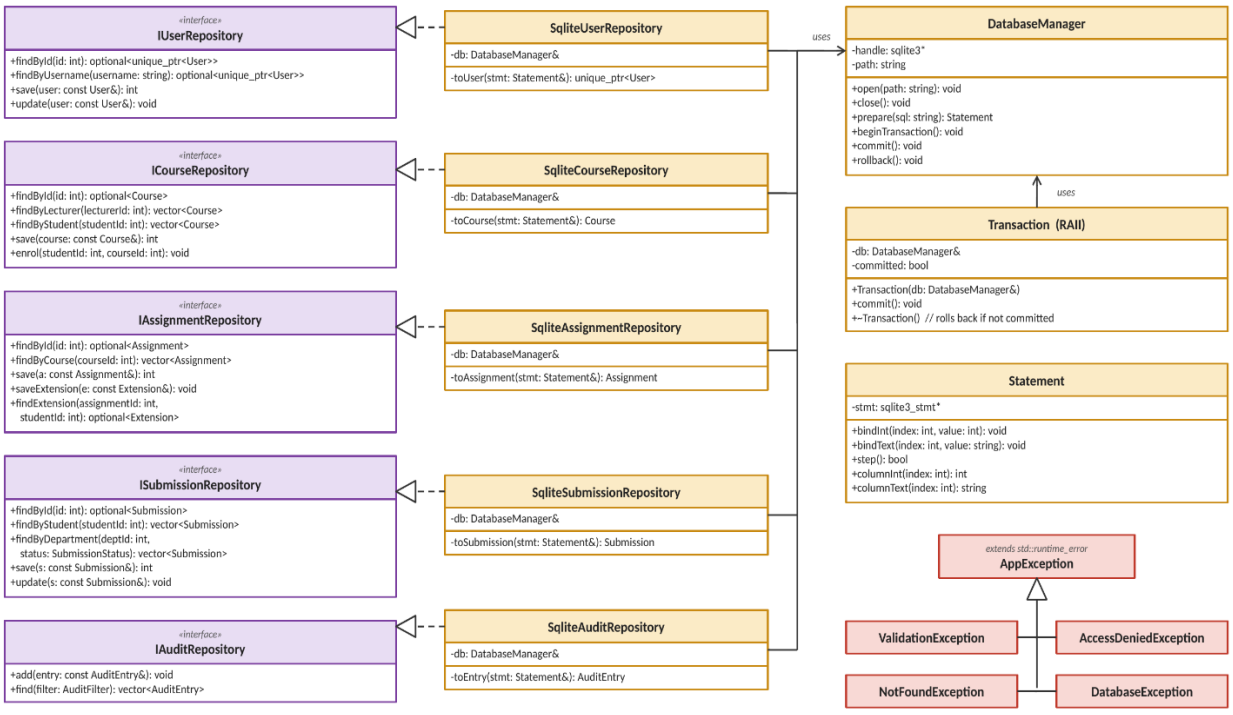


Visibility: + public, - private. Italic names are abstract. Filled diamond: the part cannot exist without the whole. Other entities refer to Student and Lecturer by id (for example Extension.studentId).

Business layer, Strategy pattern and utilities



Data access layer, Repository pattern and exceptions



User is abstract. Administrator, Lecturer, Head of Department and Student inherit from it and override *getRole()* and *AllowedActions().* This means the rest of the system can treat any user in the same way, and each role behaves correctly (polymorphism)

b. Design Pattern

Strategy: *LatePenaltyPolicy. MarkingService* asks a *LatePenaltyPolicy* for the penalty, and *TenPercentPerDayPolicy* is the rule the school uses today (10% per day for up to three days, then zero). I chose Strategy because the late rule is the rule most likely to change and the one that needs the most testing. As a separate class it can be tested on its own with boundary values, and a different rule can be added without changing *MarkingService*.

Repository: the five repository interfaces and their SQLite classes. The services use interfaces such as *IUserRepository,* and the *SQLite Repository* classes implement them. I chose Repository because it keeps all SQL in one place (NFR4), lets the services be tested with fake in-memory repositories (NFR10), and means a different database could be used later by writing new implementations.

Factory: UserFactory. Users are stored with a role, and when a user is loaded the system must create the right kind of object (*Administrator, Lecturer, HeadOfDepartment or Student*). I chose a Factory so that this decision is made in one place, and no other code needs to check the role to create a user.

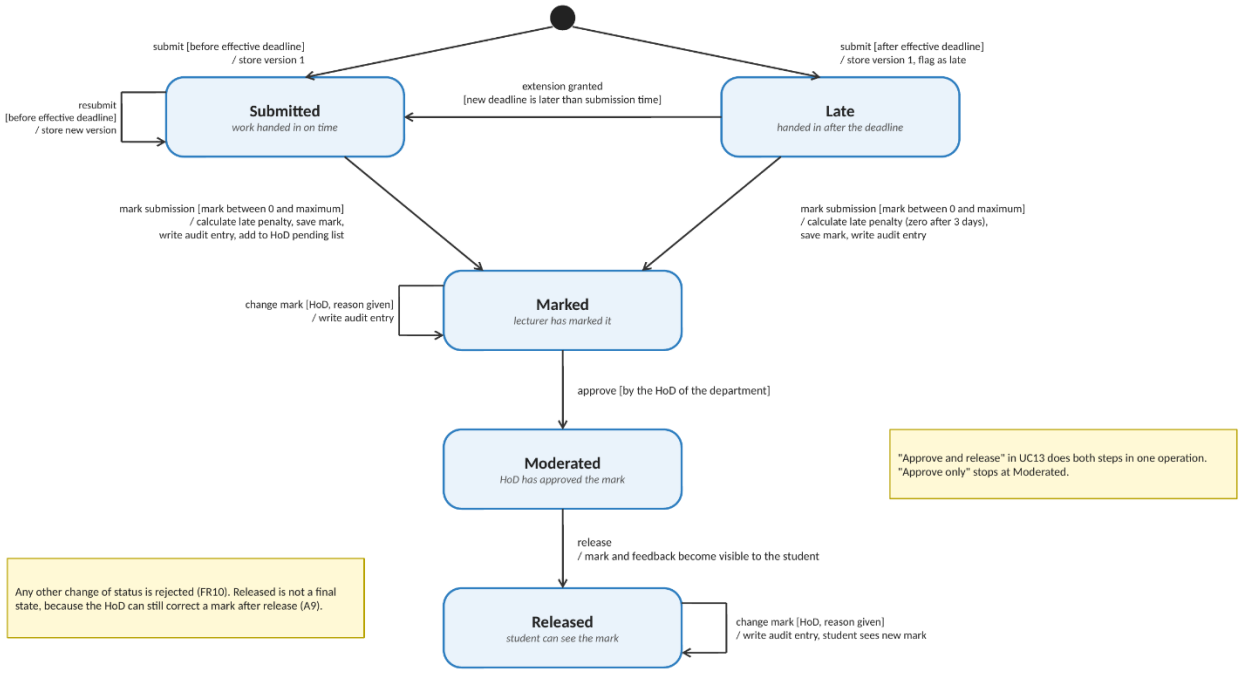
c. SOLID Principles

- **Single responsibility:** each class has one job. *MarkingService* marks submissions, *AuditService* writes the audit log, and *PasswordHasher* only hashes passwords.
- **Open/closed:** a new late-penalty rule or a new repository implementation is added as a new class. The existing services do not need to change.
- **Liskov substitution:** any *User* subclass can be used where a *User* is expected, and any repository implementation can replace another without the services noticing.
- **Interface segregation:** there are five small repository interfaces and not one large one, so each service depends only on the methods it uses. For example*, AuditService* only needs *IAuditRepository*
- **Dependency inversion:** services depend on interfaces (the repositories and *latepenaltypolicy*), not on SQLite classes. The SQLite classes depend on the same interfaces.

3. BEHAVIOURAL MIODELS

(i) State Machine for Submission Lifecycle

Submission life cycle



A state machine shows the states one object can be in and the events that move it from one state to another. Here the object is a Submission. Each arrow is written as **event [guard] / action**: the event that happens, the condition that must be true, and what the system does.

States

- **Submitted:** the work was handed in on or before the effective deadline.
- **Late:** the work was handed in after the effective deadline. It is accepted and flagged, never rejected (BR2).
- **Marked:** the lecturer has entered a mark and feedback, and the system has calculated the late penalty. The submission now appears in the HoD's list.
- **Moderated:** the HoD has approved the mark. Students still cannot see it.
- **Released:** the mark and feedback are visible to the student (BR1).

Transitions

- A new submission starts in **Submitted** if it is handed in before the effective deadline, or in **Late** if it is handed in after it.
- **Submitted to Submitted:** the student resubmits before the effective deadline. A new version is stored and the earlier versions are kept (BR5).
- **Late to Submitted:** the lecturer grants an extension and the new deadline is later than the time the work was handed in. The work is no longer late (UC9, alternative flow A1).

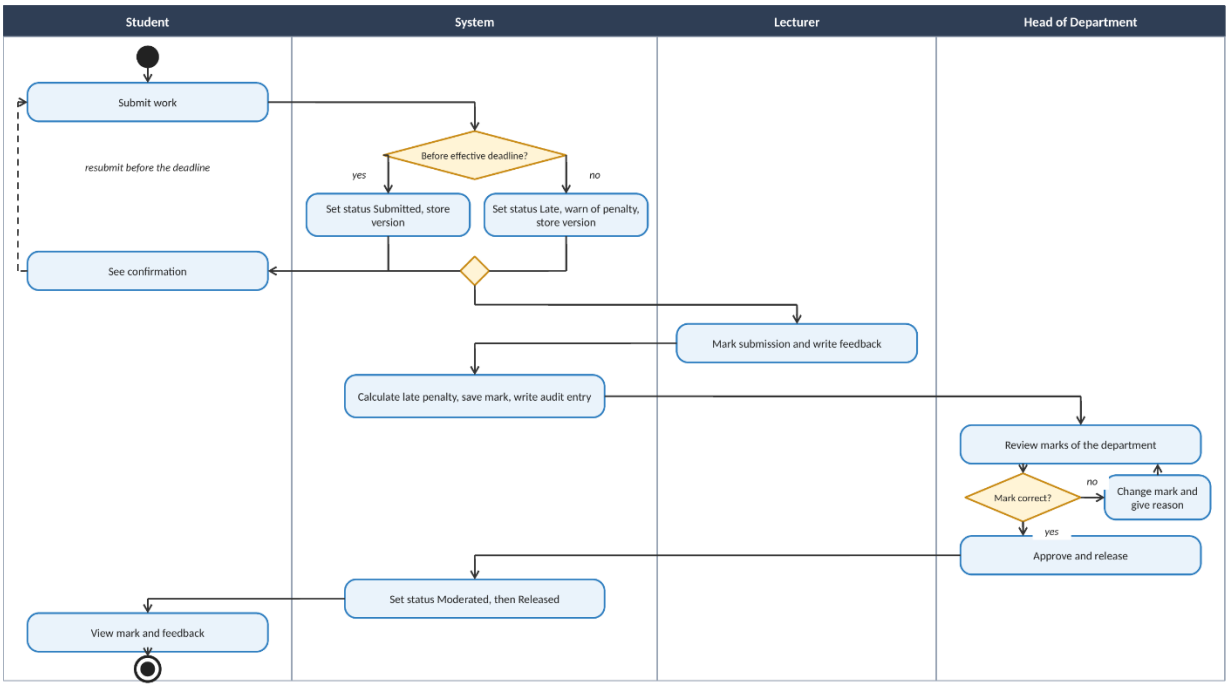
- **Submitted or Late to Marked:** the lecturer marks the submission with a mark between zero and the maximum. The system calculates the late penalty, saves the mark and writes an audit entry.
- **Marked to Marked:** the HoD changes the mark and gives a reason. The audit log records the change.
- **Marked to Moderated:** the HoD approves the mark. Only the HoD of the submission's department can do this (BR6).
- **Moderated to Released:** the HoD releases the mark, so the student can see it.
- **Released to Released:** the HoD corrects a mark after release (A9). The reason is required, the change is logged, and the student sees the new mark.

Rules the diagram shows

- Any change of status that is not on the diagram is rejected (FR10). For example, a Released submission cannot go back to Submitted.
- Released is not a final state, because the HoD can still correct a mark.
- The Submission class enforces this with canTransitionTo() and transitionTo().
- In UC13, "Approve and release" performs the Marked to Moderated and Moderated to Released steps in one operation. "Approve only" stops at Moderated.

(ii) Activity Diagram with Swimlanes: From submission to released mark

Activity diagram with swimlanes



An activity diagram shows the steps of a process. The four swimlanes show who does each step: the Student, the System, the Lecturer and the Head of Department. The solid circle is the start, the circle with a dot is the end, and diamonds are decisions.

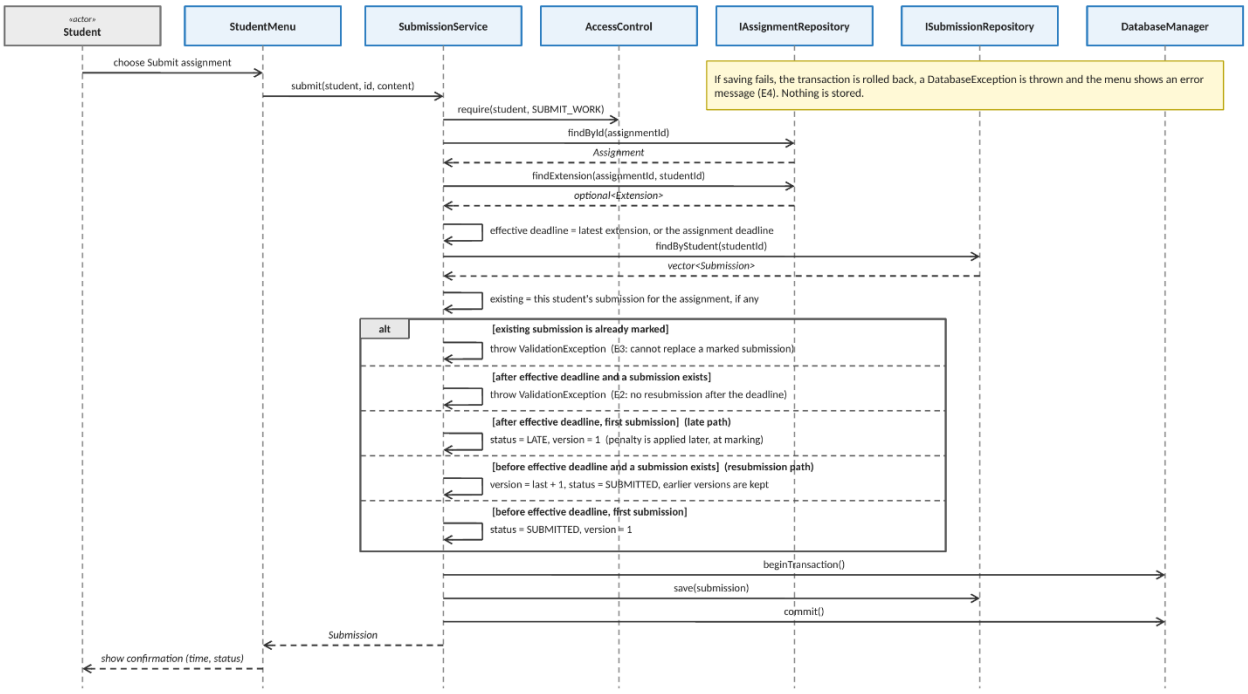
1. The **student** submits work.
2. The **system** decides whether the effective deadline has passed. If not, it sets the status to Submitted. If it has, it sets the status to Late and warns the student about the penalty. Either way it stores the version.
3. The **student** sees a confirmation. The dashed arrow shows that the student can resubmit before the deadline, which returns to step 1.
4. The **lecturer** marks the submission and writes feedback.
5. The **system** calculates the late penalty, saves the mark and writes an audit entry.
6. The **HoD** reviews the marks of the department.
7. If a mark is not correct, the HoD changes it and gives a reason, then reviews again. If it is correct, the HoD approves and releases it.
8. The **system** sets the status to Moderated and then Released.
9. The **student** views the mark and feedback. The process ends.

The diagram shows that a student can never see a mark before the HoD has released it, because the student's last step comes only after the HoD's approval (C1, BR1).

(iii) Sequence Diagrams

➢ Submit Assignment(UC17)

Submit assignment, including the late and resubmission



A sequence diagram shows which objects send which messages, in time order from top to bottom. Solid arrows are calls and dashed arrows are returns. The boxes marked **alt** show alternative paths, where only the branch whose condition is true is executed.

Main steps

1. The student chooses Submit assignment, and StudentMenu calls SubmissionService.submit().
2. The service asks AccessControl to check the student's role. This is the first thing every service does (NFR5).
3. It loads the assignment and looks for an extension, then works out the effective deadline: the latest extension if there is one, otherwise the original deadline (BR3).
4. It loads the student's submissions and finds the one for this assignment, if any.
5. The **alt** box chooses what to do, using two facts: is the effective deadline past, and does a submission already exist?
6. The service begins a transaction, saves the submission with its new version, and commits.
7. The menu shows a confirmation with the time and status.

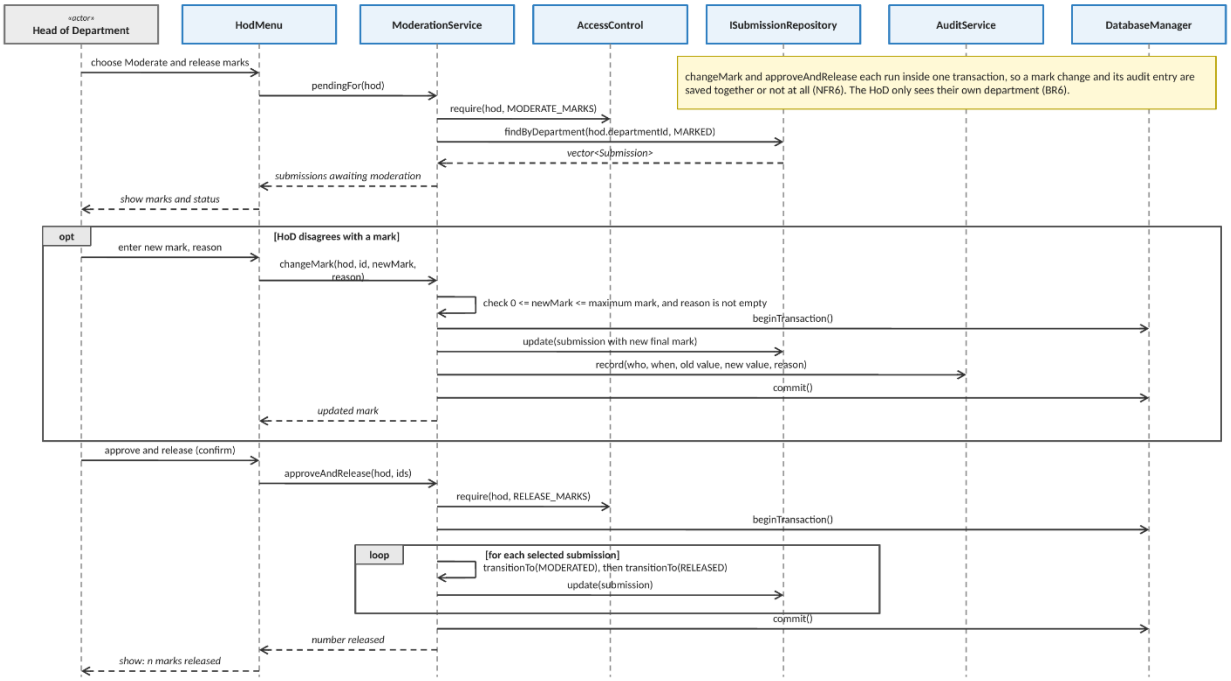
The five paths in the alt box

- **Already marked:** the submission cannot be replaced, so a ValidationException is thrown (exception flow E3).
- **After the deadline with an existing submission:** resubmission is refused (E2, BR5).
- **After the deadline, first submission (late path):** the status is LATE and the version is 1. The penalty is not calculated yet. It is calculated later, at marking.
- **Before the deadline with an existing submission (resubmission path):** the version is the last one plus one, the status stays SUBMITTED, and the earlier versions are kept.
- **Before the deadline, first submission:** the status is SUBMITTED and the version is 1.

Why a transaction. The save is wrapped in a transaction so that nothing is half-stored. If saving fails, the transaction is rolled back, a DatabaseException is thrown and the menu shows an error message without crashing (E4, NFR6).

➢ Moderate and Release Marks

Moderate and release marks



Main steps

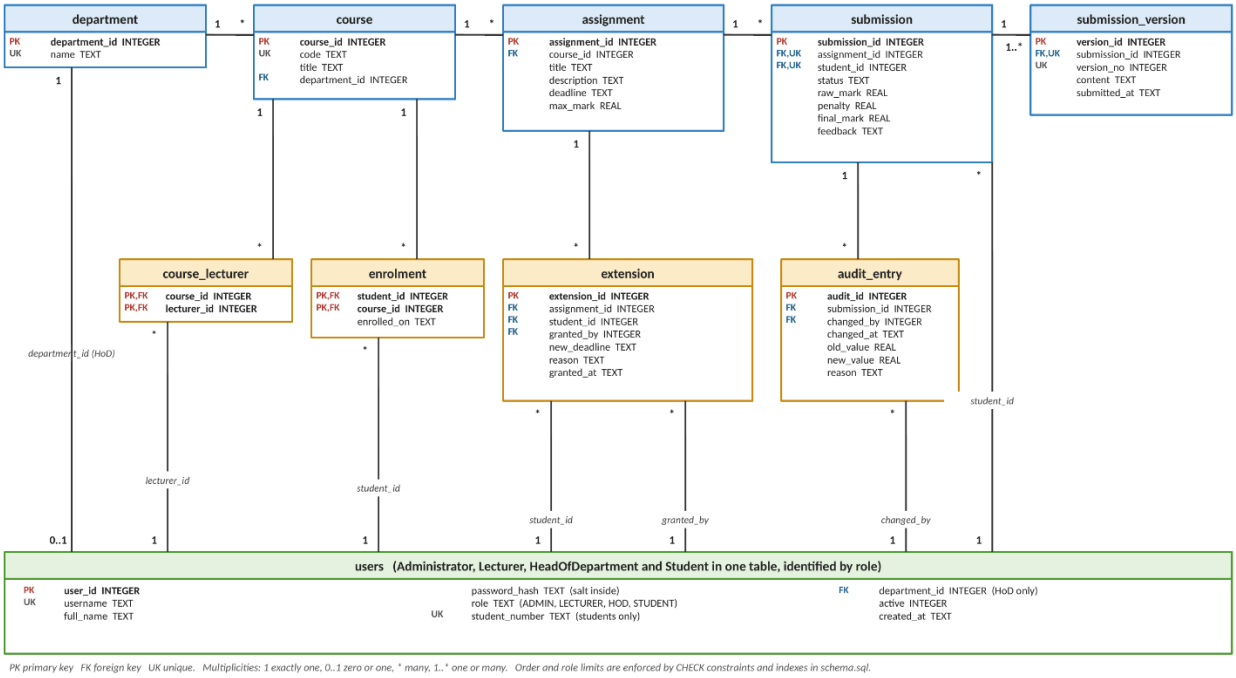
1. The HoD chooses Moderate and release marks, and HodMenu calls ModerationService.pendingFor().
2. The service checks the role, then loads only the submissions in the HoD's own department that have the status MARKED. This is how the HoD is limited to their department (BR6).
3. The menu shows the marks, penalties, feedback and status.
4. **Opt box, only if the HoD disagrees with a mark:** the HoD enters a new mark and a reason, and changeMark() checks that the mark is between zero and the maximum and that the reason is not empty. Inside one transaction the service updates the submission and asks AuditService to record who, when, the old value, the new value and the reason. Then it commits.
5. The HoD approves and releases, and approveAndRelease() checks the role again.
6. Inside one transaction, the **loop box** takes each selected submission through MODERATED and then RELEASED, and saves it. Then the service commits.
7. The menu shows how many marks were released.

Why one transaction. In changeMark() the mark update and its audit entry are committed together, so there is never a changed mark without its record (NFR6). In approveAndRelease() all the selected submissions are released together or none are.

4. DATABASE DESIGN

A. Entity Relationship Diagram

Figure 10: ERD for the CampusNode LMS (paste Database_ERD.png here).



The database has ten tables. The relationships are:

- A **department** has many **courses**, and at most one Head of Department (a row in **users**).
- A **course** is taught by many lecturers and has many students. These many-to-many relationships are stored in the join tables **course_lecturer** and **enrolment**.
- A **course** has many **assignments**.
- An **assignment** has many **extensions** and many **submissions** (one per student).
- A **submission** has one or more **submission_versions** and many **audit_entries**.
- **users** is referred to from the join tables, extensions, submissions and the audit log, because people appear in all of them as students, lecturers or the person who made a change.

B. Normalization

The design is in third normal form (3NF).

- **First normal form:** every column holds one value, and there are no repeating groups. For example, the versions of a submission are rows in `submission_version`, not columns such as `version1` and `version2`. Lecturers of a course and students of a course are also rows in their own tables.
- **Second normal form:** every column depends on the whole key. Only `course_lecturer` and `enrolment` have a combined key. The one extra column, `enrolled_on`, depends on both the student and the course. All other tables have a single-column key.
- **Third normal form:** no column depends on another column that is not the key. Each fact is stored once. For example, `course` stores `department_id` and not the department name, `submission` stores `student_id` and not the student's name, and `assignment` stores `course_id` and not the course title.

Decisions to explain

- **The effective deadline is not stored.** It is calculated from `assignment.deadline` and the latest `extension.new_deadline`, so there is no second copy that could disagree.
- **`Penalty` and `final mark` are stored on purpose.** The penalty must stay as it was when the work was marked, even if the school changes the late rule later. The final mark is not always raw mark minus penalty, because the HoD can change it, so it is its own fact.
- **`Status` is stored.** It is a workflow state changed by actions (marking, approval, release), not something that can be calculated.
- **`Users` is one table with some empty columns.** `student_number` is used only by students, and `department_id` only by Heads of Department. Each depends only on `user_id`, so this is still 3NF. CHECK constraints keep it correct: a student must have a student number, a HoD must have a department, and a department can have only one HoD.

C. Data Dictionary

Types are SQLite types. Dates and times are stored as text in the form `YYYY-MM-DD HH:MM:SS`. Key: PK primary key, FK foreign key, UK unique.

Table 1: department

Column	Type	Key	Null	Description
department_id	INTEGER	PK	No	Unique id, generated automatically
name	TEXT	UK	No	Department name

Table 2: users

Column	Type	Key	Null	Description

Column	Type	Key	Null	Description
user_id	INTEGER	PK	No	Unique id, generated automatically
username	TEXT	UK	No	Login name
full_name	TEXT		No	Full name
password_hash	TEXT		No	Salted hash of the password. The salt is inside the string. The password itself is never stored.
role	TEXT		No	ADMIN, LECTURER, HOD or STUDENT
student_number	TEXT	UK	Yes	Students only
department_id	INTEGER	FK	Yes	Heads of Department only
active	INTEGER		No	1 active, 0 deactivated. Default 1.
created_at	TEXT		No	When the account was created

Table 3: course

Column	Type	Key	Null	Description
course_id	INTEGER	PK	No	Unique id
code	TEXT	UK	No	Course code
title	TEXT		No	Course title
department_id	INTEGER	FK	No	Department the course belongs to

Table 4: course_lecturer

Column	Type	Key	Null	Description
course_id	INTEGER	PK, FK	No	The course
lecturer_id	INTEGER	PK, FK	No	A lecturer (a row in users) who teaches the course

Table 5: enrolment

Column	Type	Key	Null	Description
student_id	INTEGER	PK, FK	No	The student (a row in users)
course_id	INTEGER	PK, FK	No	The course
enrolled_on	TEXT		No	Date of enrolment

Table 6: assignment

Column	Type	Key	Null	Description

Column	Type	Key	Null	Description
assignment_id	INTEGER	PK	No	Unique id
course_id	INTEGER	FK	No	Course the assignment belongs to
title	TEXT		No	Title
description	TEXT		Yes	Description
deadline	TEXT		No	Original deadline
max_mark	REAL		No	Maximum mark, must be greater than 0

Table 7: extension

Column	Type	Key	Null	Description
extension_id	INTEGER	PK	No	Unique id
assignment_id	INTEGER	FK	No	The assignment
student_id	INTEGER	FK	No	The student who got the extension
granted_by	INTEGER	FK	No	The lecturer who granted it
new_deadline	TEXT		No	The extended deadline
reason	TEXT		No	Written reason, must not be blank
granted_at	TEXT		No	When it was granted

Table 8: submission

Column	Type	Key	Null	Description
submission_id	INTEGER	PK	No	Unique id
assignment_id	INTEGER	FK, UK	No	The assignment. Together with student_id it is unique, so a student has one submission per assignment.
student_id	INTEGER	FK, UK	No	The student
status	TEXT		No	SUBMITTED, LATE, MARKED, MODERATED or RELEASED
raw_mark	REAL		Yes	The lecturer's mark before the penalty
penalty	REAL		No	Marks taken off for lateness, calculated at marking. Default 0.
final_mark	REAL		Yes	The mark the student receives. Raw mark minus penalty, or the mark the HoD set.

Column	Type	Key	Null	Description
feedback	TEXT		Yes	Written feedback

Table 9: submission_version

Column	Type	Key	Null	Description
version_id	INTEGER	PK	No	Unique id
submission_id	INTEGER	FK, UK	No	The submission. Together with version_no it is unique.
version_no	INTEGER	UK	No	1, 2, 3 and so on
content	TEXT		No	The submitted text or file path
submitted_at	TEXT		No	When this version was handed in

Table 10: audit_entry

Column	Type	Key	Null	Description
audit_id	INTEGER	PK	No	Unique id
submission_id	INTEGER	FK	No	The submission whose mark changed
changed_by	INTEGER	FK	No	Who made the change
changed_at	TEXT		No	When
old_value	REAL		Yes	Mark before the change. Empty for the first mark.
new_value	REAL		No	Mark after the change
reason	TEXT		No	Why, must not be blank

D. SQL Schema

The program must run `PRAGMA foreign_keys = ON;` on every new connection, because SQLite does not check foreign keys unless this is switched on.

-- CampusNode LMS database schema (SQLite)

-- The program must run "PRAGMA foreign_keys = ON;" on every new connection.

```
-- Dates and times are stored as ISO-8601 text: 'YYYY-MM-DD HH:MM:SS'

PRAGMA foreign_keys = ON;

CREATE TABLE department
(
    department_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name          TEXT NOT NULL UNIQUE
);

CREATE TABLE users
(
    user_id         INTEGER PRIMARY KEY AUTOINCREMENT,
    username        TEXT NOT NULL UNIQUE,
    full_name       TEXT NOT NULL,
    password_hash   TEXT NOT NULL,
    role            TEXT NOT NULL CHECK (role IN ('ADMIN', 'LECTURER', 'HOD',
'STUDENT')),
    student_number  TEXT UNIQUE,
    department_id   INTEGER REFERENCES department (department_id),
    active          INTEGER NOT NULL DEFAULT 1 CHECK (active IN (0, 1)),
    created_at      TEXT NOT NULL DEFAULT (datetime('now')),
    CHECK ((role = 'STUDENT') = (student_number IS NOT NULL)),
    CHECK ((role = 'HOD') = (department_id IS NOT NULL))
);
CREATE UNIQUE INDEX idx_one_hod_per_department ON users (department_id) WHERE role =
'HOD';

CREATE TABLE course
(
    course_id      INTEGER PRIMARY KEY AUTOINCREMENT,
    code           TEXT NOT NULL UNIQUE,
    title          TEXT NOT NULL,
    department_id  INTEGER NOT NULL REFERENCES department (department_id)
);
CREATE INDEX idx_course_department ON course (department_id);

CREATE TABLE course_lecturer (
    course_id    INTEGER NOT NULL REFERENCES course (course_id),
    lecturer_id  INTEGER NOT NULL REFERENCES users (user_id),
    PRIMARY KEY (course_id, lecturer_id)
```

```
);

CREATE TABLE enrolment (
    student_id   INTEGER NOT NULL REFERENCES users (user_id),
    course_id    INTEGER NOT NULL REFERENCES course (course_id),
    enrolled_on  TEXT NOT NULL DEFAULT (date('now')),
    PRIMARY KEY (student_id, course_id)
);
CREATE INDEX idx_enrolment_course ON enrolment (course_id);

CREATE TABLE assignment
(
    assignment_id  INTEGER PRIMARY KEY AUTOINCREMENT,
    course_id      INTEGER NOT NULL REFERENCES course (course_id),
    title          TEXT NOT NULL,
    description    TEXT,
    deadline       TEXT NOT NULL,
    max_mark       REAL NOT NULL CHECK (max_mark > 0)
);
CREATE INDEX idx_assignment_course ON assignment (course_id);

CREATE TABLE extension
(
    extension_id   INTEGER PRIMARY KEY AUTOINCREMENT,
    assignment_id  INTEGER NOT NULL REFERENCES assignment (assignment_id),
    student_id     INTEGER NOT NULL REFERENCES users (user_id),
    granted_by     INTEGER NOT NULL REFERENCES users (user_id),
    new_deadline   TEXT NOT NULL,
    reason         TEXT NOT NULL CHECK (length(trim(reason)) > 0),
    granted_at     TEXT NOT NULL DEFAULT (datetime('now'))
);
CREATE INDEX idx_extension_lookup ON extension (assignment_id, student_id);

CREATE TABLE submission
(
    submission_id  INTEGER PRIMARY KEY AUTOINCREMENT,
    assignment_id  INTEGER NOT NULL REFERENCES assignment (assignment_id),
    student_id     INTEGER NOT NULL REFERENCES users (user_id),
    status         TEXT NOT NULL CHECK (status IN ('SUBMITTED', 'LATE', 'MARKED',
'MODERATED', 'RELEASED')),
    raw_mark       REAL CHECK (raw_mark IS NULL OR raw_mark >= 0),
    penalty        REAL NOT NULL DEFAULT 0 CHECK (penalty >= 0),
    final_mark     REAL CHECK (final_mark IS NULL OR final_mark >= 0),
    feedback       TEXT,
    UNIQUE (assignment_id, student_id),
    CHECK ((status IN ('SUBMITTED', 'LATE')) = (raw_mark IS NULL AND final_mark IS
NULL))
);
CREATE INDEX idx_submission_student ON submission (student_id);
CREATE INDEX idx_submission_status ON submission (status);

CREATE TABLE submission_version
(
    version_id     INTEGER PRIMARY KEY AUTOINCREMENT,
    submission_id  INTEGER NOT NULL REFERENCES submission (submission_id),
```

```
    version_no     INTEGER NOT NULL CHECK (version_no >= 1),
    content        TEXT NOT NULL CHECK (length(content) > 0),
    submitted_at   TEXT NOT NULL,
    UNIQUE (submission_id, version_no)
);

CREATE TABLE audit_entry
(
    audit_id       INTEGER PRIMARY KEY AUTOINCREMENT,
    submission_id  INTEGER NOT NULL REFERENCES submission (submission_id),
    changed_by     INTEGER NOT NULL REFERENCES users (user_id),
    changed_at     TEXT NOT NULL DEFAULT (datetime('now')),
    old_value      REAL,
    new_value      REAL NOT NULL,
    reason         TEXT NOT NULL CHECK (length(trim(reason)) > 0)
);
CREATE INDEX idx_audit_submission ON audit_entry (submission_id);
CREATE INDEX idx_audit_changed_at ON audit_entry (changed_at);

CREATE TRIGGER audit_entry_no_update BEFORE UPDATE ON audit_entry
BEGIN SELECT RAISE(ABORT, 'audit log is append-only'); END;

CREATE TRIGGER audit_entry_no_delete BEFORE DELETE ON audit_entry
BEGIN SELECT RAISE(ABORT, 'audit log is append-only'); END;
```

What the constraints protect

- **Keys and foreign keys** stop rows pointing at things that do not exist.
- **CHECK constraints** limit roles and statuses to the allowed values, keep marks at zero or above, and stop blank reasons.
- **The submission check** means a submission has marks exactly when it is Marked, Moderated or Released.
- **The unique pair (assignment_id, student_id)** gives each student one submission per assignment. Resubmissions become new versions.
- **The triggers** make the audit log append-only, so an entry can be added but never changed or deleted (SRS section on data requirements).
- **Indexes** on the columns used to look rows up keep login and list screens fast with 800 students (NFR1, NFR2).

The script was run against SQLite and the constraints and triggers were tested: a second HoD for the same department, a student without a student number, a marked submission without a mark, a blank reason and any change to the audit log were all rejected.

E. Mapping Between Tables and Classes

How the class diagram maps to the tables

Class	Table	Difference and reason
User, Administrator, Lecturer, HeadOfDepartment, Student	users	The inheritance is flattened into one table with a `role` column. The roles differ by only one attribute each, so one table makes login and lookups simple. `departmentId` and `studentNumber` become columns that are empty for the other roles, protected by CHECK constraints.
Department	department	Direct.
Course	course	Direct. The `department_id` column is the link to Department.
(the Lecturer and Course association)	course_lecturer	A many-to-many association has no class in the diagram but needs a join table in a database.
Enrolment	enrolment	The association class becomes a table whose key is the student and the course.
Assignment	assignment	Direct.
Extension	extension	Direct. `studentId` and `grantedById` become the foreign keys `student_id` and `granted_by`.
Submission	submission	Direct. The enumeration `SubmissionStatus` becomes a text column with a CHECK constraint.
SubmissionVersion	submission_version	Direct. It is a separate table because a submission has many versions.
AuditEntry	audit_entry	Direct. `changedById` becomes `changed_by`. Triggers make it append-only.

Other differences

- **Classes with no table:** the services, repositories, `LatePenaltyPolicy`, `UserFactory`, `AccessControl`, `PasswordHasher` and the exceptions describe behaviour, not stored data.
- **Enumerations:** `Role` and `SubmissionStatus` are not tables. SQLite has no enumeration type, and there are only a few fixed values, so a CHECK constraint is simpler.
- **Types:** `Date` and `DateTime` are stored as text in ISO format, `bool` as 0 or 1, and `optional<double>` as a REAL column that may be empty.
- **Names:** class attributes in camelCase become table columns in snake_case.

- **Password:** the class attribute `passwordHash` is stored in `password_hash`. It is a salted hash, and the salt is stored inside the same string.